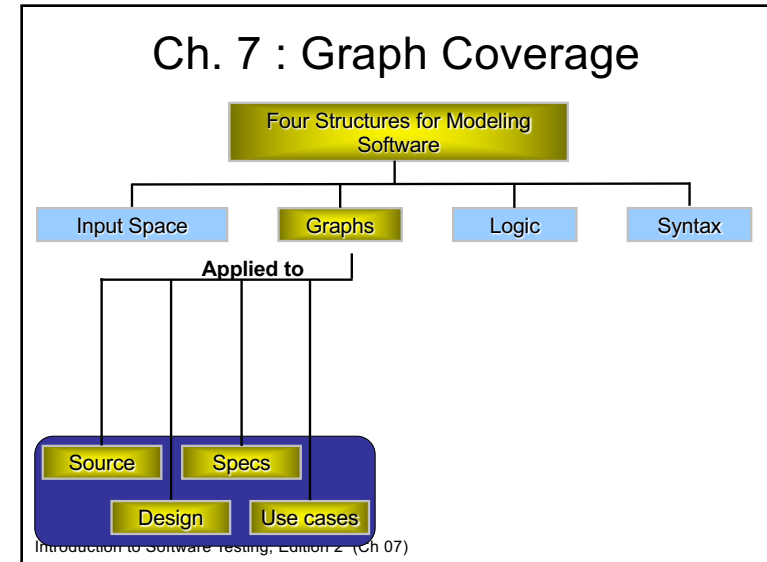


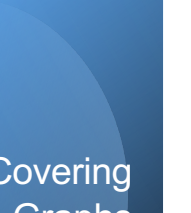
COMS/SE 4170
Software Testing
Spring 2026

Graph-based Testing

1



2



Covering Graphs

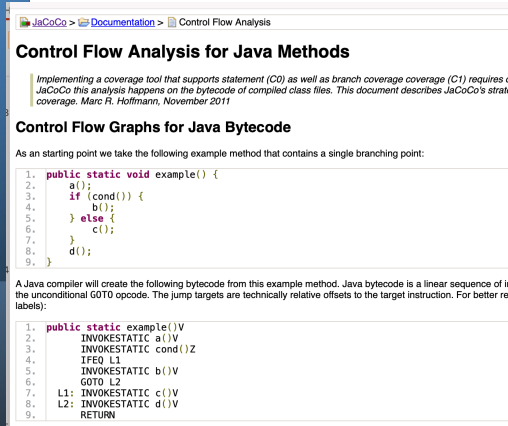
- Graphs are the **most commonly used structure** for testing
- Graphs can come from **many sources**
 - Control flow graphs
 - Design structure
 - FSMs and statecharts
 - Use cases
 - Interfaces (GUIs)
- Tests usually are intended to “cover” the graph in some way

Introduction to Software Testing, Edition 2 (Ch 07)

3



Covering Graphs



Introduction to Software Testing, Edition 2 (Ch 07)

4

Definition of a Graph

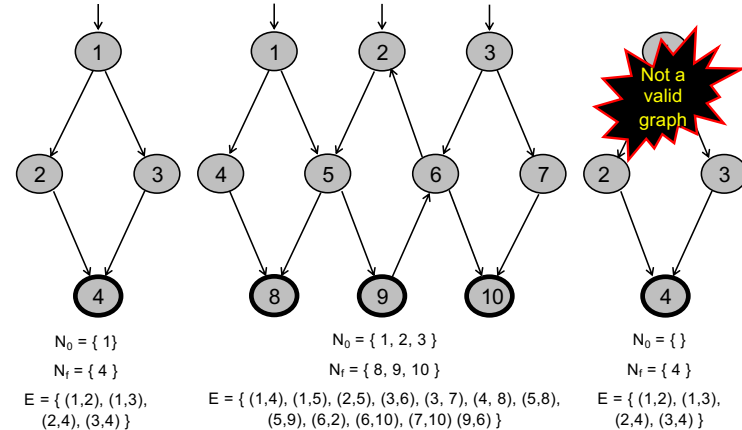
- A set N of nodes, N is not empty
- A set N_0 of initial nodes, N_0 is not empty
- A set N_f of final nodes, N_f is not empty
- A set E of edges, each edge from one node to another
 - (n_i, n_j) , i is predecessor, j is successor

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

5

Example Graphs



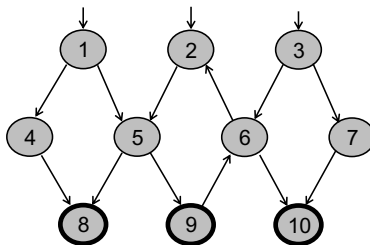
Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

6

Paths in Graphs

- **Path:** A sequence of nodes – $[n_1, n_2, \dots, n_M]$
 - Each pair of nodes is an edge
- **Length:** The number of edges
 - A single node is a path of length 0
- **Subpath:** A subsequence of nodes in p is a subpath of p



A Few Paths

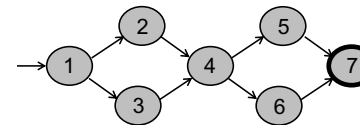
[1, 4, 8]
[2, 5, 9, 6, 2]
[3, 7, 10]

Introduction to Software Testing, Edition 2 (Ch 07)

7

Test Paths and SESEs

- **Test Path:** A path that starts at an initial node and ends at a final node
- Test paths represent execution of test cases
 - Some test paths can be executed by many tests
 - Some test paths cannot be executed by any tests
- **SESE graphs:** All test paths start at a single node and end at another node
 - Single-entry, single-exit (SESE)
 - N_0 and N_f have exactly one node



Double-diamond graph

Four test paths
[1, 2, 4, 5, 7]
[1, 2, 4, 6, 7]
[1, 3, 4, 5, 7]
[1, 3, 4, 6, 7]

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

8

Visiting and Touring

- Visit : A test path p **visits node n** if n is in p
A test path p **visits edge e** if e is in p
- Tour : A test **path p tours subpath q** if q is a subpath of p

Path [1, 2, 4, 5, 7]

Visits nodes 1, 2, 4, 5, 7

Visits edges (1, 2), (2, 4), (4, 5), (5, 7)

Tours subpaths [1, 2, 4], [2, 4, 5], [4, 5, 7], [1, 2, 4, 5], [2, 4, 5, 7], [1, 2, 4, 5, 7]

(Also, each edge is technically a subpath)

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

9

Tests and Paths

- Test criteria **may require that a test starts on one node and ends at another**
- Sometimes a **path** on a graph **cannot actually be executed on a program**:
 - e.g. execute a loop zero times, but the program always executes at least once (based on semantics of program)

10

Tests and Test Paths

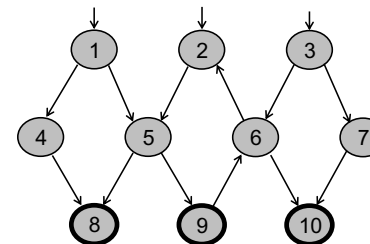
- **path (t)**: The **test path** executed by test t
- **path (T)**: The **set of test paths** executed by the set of tests T
- Each test executes one and only one test path
 - Complete execution from a start node to a final node
- A location in a graph (node or edge) **can be reached** from another location if there is **a sequence of edges from the first location to the second**
 - **Syntactic reach**: A subpath exists in the graph
 - **Semantic reach**: A test exists that can execute that subpath
 - This distinction will become important in section 7.3

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

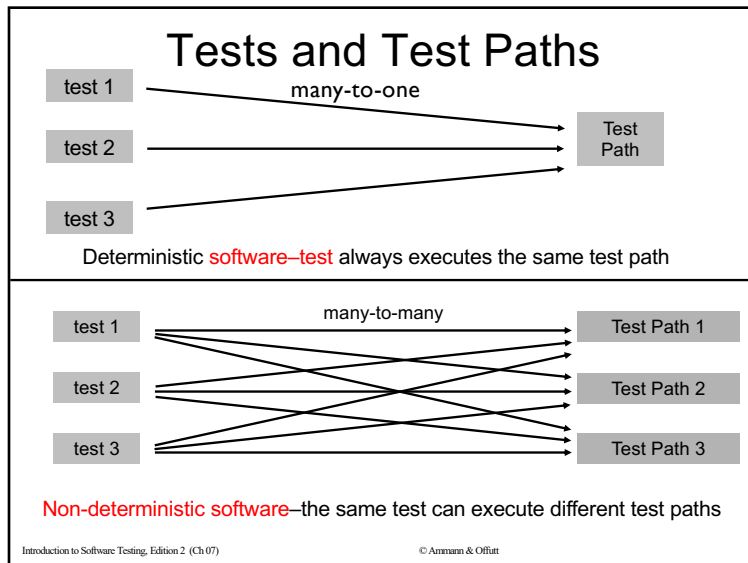
11

Reachability of Nodes



Can reach all nodes
except 3 and 7 from Node 1

12



13

Testing and Covering Graphs

- We use graphs in testing as follows :
 - Develop a **model of the software** as a graph
 - Require tests to **visit or tour specific sets of nodes, edges or subpaths**

Introduction to Software Testing, Edition 2 (Ch 07) © Ammann & Offutt

14

Testing and Covering Graphs

- Test Requirements (TR):** Describe properties of test paths
- Test Criterion : Rules that define test requirements
- Satisfaction:** *Given a set TR of test requirements for a criterion C, a set of tests T satisfies C on a graph if and only if for every test requirement in TR, there is a test path in path(T) that meets the test requirement tr*
- Structural Coverage Criteria:** Defined on a graph just in terms of nodes and edges
- Data Flow Coverage Criteria:** Requires a graph to be annotated with references to variables

Introduction to Software Testing, Edition 2 (Ch 07) © Ammann & Offutt

15

Node and Edge Coverage

- The first (and simplest) two criteria require that each node and edge in a graph be executed

Node Coverage (NC): Test set T satisfies node coverage on graph G iff for every syntactically reachable node n in N , there is some path p in $path(T)$ such that p visits n .

- More simply stated:

Node Coverage (NC) : TR contains each reachable node in G .

Introduction to Software Testing, Edition 2 (Ch 07) © Ammann & Offutt

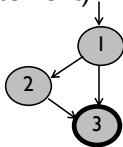
16

Node and Edge Coverage

- Edge coverage is slightly stronger than node

Edge Coverage (EC) : TR contains each reachable path of length up to 1, inclusive, in G.

- The phrase “length up to 1” allows for graphs with one node and no edges
- NC and EC are only different when there is an edge and another subpath between a pair of nodes (as in an “if-else” statement)



Node Coverage : TR = { 1, 2, 3 }
Test Path = [1, 2, 3]

Edge Coverage : TR = { (1, 2), (1, 3), (2, 3) }
Test Paths = [1, 2, 3]
[1, 3]

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

17

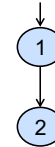
Paths of Length 1 and 0

- A graph with only one node will not have any edges



- It may seem trivial, but formally, Edge Coverage needs to require Node Coverage on this graph
- Otherwise, Edge Coverage will not subsume Node Coverage
 - So we define “length up to 1” instead of simply “length 1”

- We have the same issue with graphs that only have one edge – for Edge-Pair Coverage ...



Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

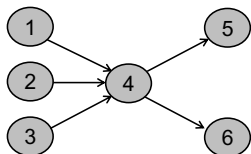
18

Covering Multiple Edges

- Edge-pair coverage requires pairs of edges, or subpaths of length 2

Edge-Pair Coverage (EPC) : TR contains each reachable path of length up to 2, inclusive, in G.

- The phrase “length up to 2” is used to include graphs that have less than 2 edges



Edge-Pair Coverage :

TR = { [1,4,5], [1,4,6], [2,4,5], [2,4,6], [3,4,5], [3,4,6] }

- The logical extension is to require all paths ...

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

19

Covering Multiple Edges

Complete Path Coverage (CPC) : TR contains all paths in G.

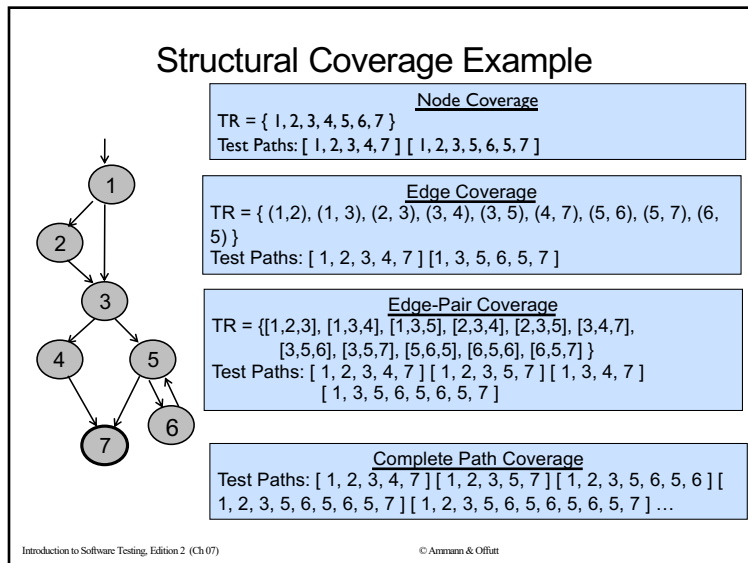
Unfortunately, this is impossible if the graph has a loop, so a weak compromise makes the tester decide which paths:

Specified Path Coverage (SPC) : TR contains a set S of test paths, where S is supplied as a parameter.

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

20



21

Handling Loops in Graphs

- If a graph contains a loop, it has an **infinite number of paths**
- Thus, **CPC (complete path coverage)** is **not feasible**
- **SPC (specified coverage)** is not satisfactory because the results are subjective and vary with the tester

Introduction to Software Testing, Edition 2 (Ch 07) © Ammann & Offutt

22

Handling Loops in Graphs

- Attempts to “deal with” loops:
 - 1970s : Execute cycles once ([4, 5, 4] in previous example, informal)
 - 1980s : Execute each loop, exactly once (formalized)
 - 1990s : Execute loops 0 times, once, more than once (informal description)
 - 2000s : Prime paths (touring, sidetrips, and detours)

Introduction to Software Testing, Edition 2 (Ch 07) © Ammann & Offutt

23

Simple Paths and Prime Paths

- **Simple Path**: A path from node n_i to n_j is **simple** if **no node appears more than once**, except possibly the first and last nodes are the same
 - No internal loops
 - A loop is a simple path
- **Prime Path**: A simple path that does not appear as a proper subpath of any other simple path

```

graph TD
    Start(( )) --> 1((1))
    1 --> 2((2))
    1 --> 3((3))
    2 --> 4((4))
    3 --> 4
    4 --> 1
  
```

Simple Paths : [1,2,4,1], [1,3,4,1], [2,4,1,2], [2,4,1,3], [3,4,1,2], [3,4,1,3], [4,1,2,4], [4,1,3,4], [1,2,4], [1,3,4], [2,4,1], [3,4,1], [4,1,2], [4,1,3], [1,2], [1,3], [2,4], [3,4], [4,1], [1], [2], [3], [4]

Prime Paths : [2,4,1,2], [2,4,1,3], [1,3,4,1], [1,2,4,1], [3,4,1,2], [4,1,3,4], [4,1,2,4], [3,4,1,3]

Introduction to Software Testing, Edition 2 (Ch 07) © Ammann & Offutt

24

Prime Path Coverage

- A simple and finite criterion that requires loops to be executed as well as skipped

Prime Path Coverage (PPC) : TR contains each prime path in G.

- Will tour all paths of length 0, 1, ...
- That is, it subsumes node and edge coverage
- PPC almost, but **not quite**, subsumes EPC ...

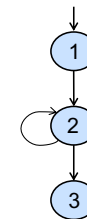
Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

25

PPC Does Not Subsume EPC

- If a **node n** has an edge to itself (**self edge**), EPC requires $[n, n, m]$ and $[m, n, n]$
- $[n, n, m]$ is not prime
- Neither $[n, n, m]$ nor $[m, n, n]$ are simple paths (not prime)



EPC Requirements :

TR = { [1,2,3], [1,2,2], [2,2,3], [2,2,2] }

PPC Requirements :

TR = { [1,2,3], [2,2] }

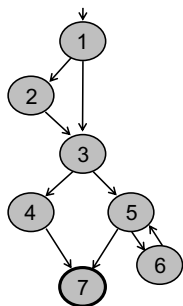
Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

26

Prime Path Example

- The previous example has 38 simple paths
- Only nine *prime paths*



Prime Paths

[1, 2, 3, 4, 7]
[1, 2, 3, 5, 7]
[1, 2, 3, 5, 6]
[1, 3, 4, 7]
[1, 3, 5, 7]
[1, 3, 5, 6]
[6, 5, 7]
[6, 5, 6]
[5, 6, 5]

Execute
loop 0 times

Execute
loop once

Execute loop
more than once

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

27

Touring, Sidetrips, and Detours

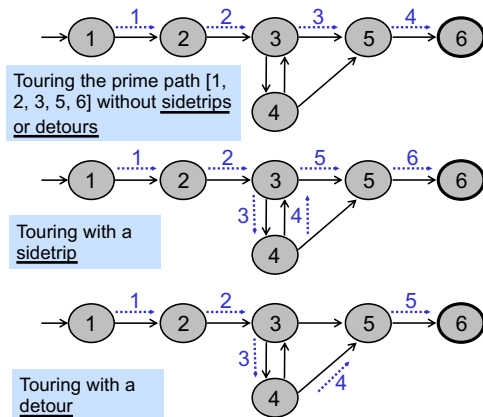
- Prime paths do not have internal loops ... test paths might
- Tour** : A test path p tours subpath q if q is a subpath of p
- Tour With Sidetrips** : A test path p tours subpath q with sidetrips iff every edge in q is also in p in the same order
 - The tour can include a sidetrip, as long as it comes back to the same node
- Tour With Detours** : A test path p tours subpath q with detours iff every node in q is also in p in the same order
 - The tour can include a detour from node ni , as long as it comes back to the prime path at a successor of ni

© Ammann & Offutt

Introduction to Software Testing, Edition 2 (Ch 07)

28

Sidetrips and Detours Example



Introduction to Software Testing, Edition 2 (Ch 07)

29

29

Infeasible Test Requirements

- An infeasible test requirement cannot be satisfied
 - Unreachable statement (dead code)
 - Subpath that can only be executed with a contradiction ($X > 0$ and $X < 0$)
- Most test criteria have **some infeasible test requirements**
- It is usually undecidable whether all test requirements are feasible**

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

30

Infeasible Test Requirements

- When sidetrips are not allowed, many structural criteria have more infeasible test requirements
- However, always allowing sidetrips weakens the test criteria

Practical recommendation—Best Effort Touring

- Satisfy as many test requirements as possible without sidetrips
- Allow sidetrips to try to satisfy remaining test requirements

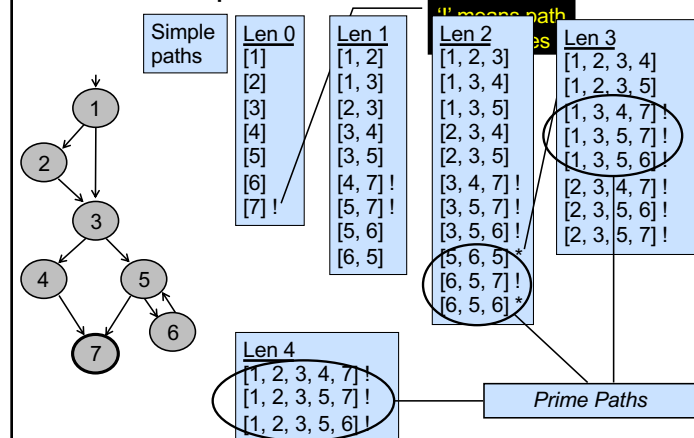
Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

31

31

Simple & Prime Path Example



Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

32

Round Trips

- **Round-Trip Path** : A prime path that starts and ends at the same node

Simple Round Trip Coverage (SRTC) : TR contains at least one round-trip path for each reachable node in G that begins and ends a round-trip path.

Complete Round Trip Coverage (CRTC) : TR contains all round-trip paths for each reachable node in G.

- These criteria omit nodes and edges that are not in round trips
- Thus, they do not subsume edge-pair, edge, or node coverage

Introduction to Software Testing, Edition 2 (Ch 07)

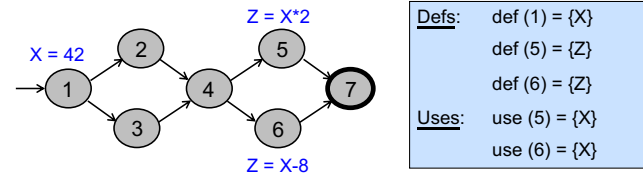
© Ammann & Offutt

33

Data Flow Criteria

Goal: Try to ensure that values are computed and used correctly

- **Def:** A location where a value for a variable is stored into memory
- **Use:** A location where a variable's value is accessed



The values given in **defs** should reach at least one, some, or all possible uses

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

34

DU Pairs and DU Paths

def (n) or def (e) : The set of variables that are **defined** by node n or edge e

use (n) or use (e) : The set of variables that **are used** by node n or edge e

DU (def-use) pair: A pair of locations (l_i, l_j) such that a variable v is defined at l_i and used at l_j

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

35

DU Pairs and DU Paths

Def-clear : A path from l_i to l_j is **def-clear with respect to variable** v if v is not given another value on any of the nodes or edges in the path

Reach : If there is a **def-clear path** from l_i to l_j with respect to v , the def of v at l_i reaches the use at l_j

du-path : A simple subpath that is def-clear with respect to v from a def of v to a use of v

- $du(n_i, n_j, v)$ – the set of du-paths from n_i to n_j
- $du(n_i, v)$ – the set of du-paths that start at n_i

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

36

Touring DU-Paths

- A test path p *du-tours* subpath d with respect to v if p tours d and the subpath taken is def-clear with respect to v
- Sidetrips can be used, just as with previous touring
- Three criteria
 - Use every def
 - Get to every use
 - Follow all du-paths

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

37

Data Flow Test Criteria

- First, we make sure every def reaches a use

All-defs coverage (ADC) : For each set of du-paths $S = du(n, v)$, TR contains at least one path d in S .

- Then we make sure that every def reaches all possible uses

All-uses coverage (AUC) : For each set of du-paths to uses $S = du(n_i, n_j, v)$, TR contains at least one path d in S .

- Finally, we cover all the paths between defs and uses

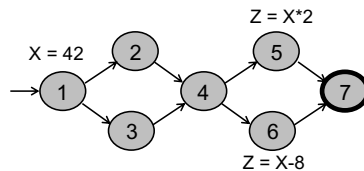
All-du-paths coverage (ADUPC): For each set $S = du(n_i, n_j, v)$, TR contains every path d in S .

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

38

Data Flow Testing Example



All-defs for X
[1, 2, 4, 5]

All-uses for X
[1, 2, 4, 5]
[1, 2, 4, 6]

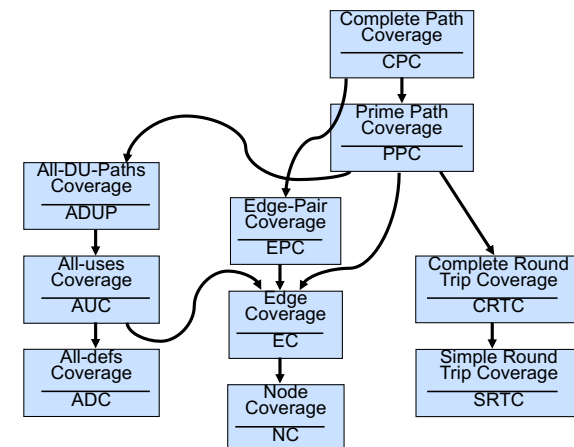
All-du-paths for X
[1, 2, 4, 5]
[1, 3, 4, 5]
[1, 2, 4, 6]
[1, 3, 4, 6]

Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

39

Graph Coverage Criteria Subsumption



Introduction to Software Testing, Edition 2 (Ch 07)

© Ammann & Offutt

40

<https://cs.gmu.edu:8443/offutt/coverage/GraphCoverage>




[Show sidebar](#)

Graph Coverage Web Application

Graph Information

Enter **center** graph edges in the test box below. For each edge enter **one** line. Enter **edges** as pairs of nodes, separated by spaces (e.g.: 1 3)

Enter **initial** nodes below (can be more than one), separated by spaces. If the test box below is empty, the first node in the left box will be the initial node.

Enter **final** nodes below (can be more than one), separated by spaces.

Test Requirements:
[Nodes](#)
[Edges](#)
[Edge-Pair](#)
[Simple Paths](#)
[Prims Paths](#)

Test Paths: Algorithm 1: Slower, more test paths, shorter test paths
 Algorithm 2: Faster, fewer test paths, longer test paths

[Node Coverage](#)
[Edge Coverage](#)
[Edge-Pair Coverage](#)
[Prims Path Coverage](#)

Algorithm 1 is our original, not particularly clever, algorithm to find test paths from graph coverage test requirements. In our 2012 ICST paper, "*Better Algorithms to Minimize the Cost of Test Paths*", we described an algorithm that combines test requirements to produce fewer, but longer test paths (algorithm 2). Users can evaluate the tradeoffs between more but shorter test paths and fewer but longer test paths and choose the appropriate algorithm.

Other Tools:
[New Graph](#)
[Data Flow Coverage](#)
[Logic Coverage](#)
[Minimal-MANUCLUT Coverage](#)

Copyright notice:
 Introduction to Software Testing, Amazon and Microsoft.
 Implementation by Paul R. Xu, Sun Li, Lin Dong, and Shoun Reem.
 © 2007-2017, all rights reserved.
 Last updated: 20-06-2017

41

Enter your **graph edges** as pairs of nodes below. Put each edge on one line. Enter edges as pairs of nodes, separated by separating a space.

1,2
2,3
3,4
2,5

Enter **initial nodes below** (can be more than one), separated by spaces. If the test box below is empty, the first node in the **Start** box will be the initial node.

Enter **final nodes below** (can be more than one), separated by spaces.

Test Requirements: Nodes Edge Edge Path Simple Paths Binary Paths

Test Suite: Algorithm 1: Shortest, more test paths, shortest test paths Node Coverage Edge Coverage Edge Path Coverage Power Path Coverage

Algorithm 2: Points, fewer test paths, longer test paths

Algorithm 1 is our original, not particularly clever, algorithm to find test paths from graph coverage test requirements. In our 2018 ICST paper, "Better algorithm to influence the test design for the Test Path", we described an alternative algorithm. This algorithm is more clever, but longer test paths algorithm. 2. Users can evaluate to whether between more shorter test paths and longer test paths and choose the appropriate algorithm.

Other Tests: Area Chart Data Flow Coverage Logic Coverage Statement Coverage

Short Graph Coverage:

28 requirements are needed for Simple Paths

Node color: **Initial Node**, **Final Node**

42

Test Requirements:

[Nodes](#)
[Edges](#)
[Edge-Pair](#)
[Simple Paths](#)
[Prime Paths](#)

Test Paths:

[Algorithm 1: Slower, more test paths, shorter test paths](#)
[Edge Coverage](#)
[Edge-Pair Coverage](#)
[Prime Path Coverage](#)

Algorithm 2: Faster, fewer test paths, longer test paths

[Edge Coverage](#)
[Edge-Pair Coverage](#)
[Prime Path Coverage](#)

Algorithm 1 is our original, not particularly clever, algorithm to find test paths from graph coverage test requirements. In our 2012 ICST paper, *"Better Algorithms Minimize the Cost of Test Paths"*, we described an algorithm that combines test requirements to produce fewer, but longer test paths (algorithm 2). Users can evaluate tradeoffs between more but shorter test paths and fewer but longer test paths and choose the appropriate algorithm.

Other Tools:

[New Graph](#)
[Data Flow Coverage](#)
[Logic Coverage](#)
[Minimal/MAXimal Coverage](#)

Share Graph:

[View](#)

9 requirements are needed for Prime Paths

- [1,2,3,4,7]
- [1,2,3,5,7]
- [1,2,3,5,6]
- [1,3,4,7]
- [1,3,5,7]
- [1,2,5,6]
- [6,5,7]
- [6,5,6]
- [5,6,5]

List any infeasible prime paths in the box below, using the numbers below the prime paths above. Separate with commas.
 Prime paths you mark as infeasible will not be used in any test paths.
 Example: 1,2,9

Node color:

Initial Node
Final Node

43

Graph Information

Please enter your graph edges in the text box below. Put each edge in one line. Enter edges as pairs of nodes, separated by spaces (e.g.: 1 3)	Please enter initial nodes below (can be more than one). If the text box below is empty, the first node in the list will be the initial node.	Enter final nodes below (can be more than one), separated by spaces.
--	---	--

Data Flow Information

Please enter your defs in the text box below. Put one variable and all defs for the variable in one line, separated by spaces. Put the variable name at the beginning of the line (e.g.: x 1 2)	Please enter your uses in the text box below. Put one variable and all uses for the variable in one line, separated by spaces. Put the variable name at the beginning of the line. (e.g.: x 3 4 2 3)
---	--

Test Requirements: DU Pairs DU Paths

Test Paths: All Def Coverage All Use Coverage All DU Path Coverage

Others: New Graph New DU Info Graph Coverage Logic Coverage Minimal-MUMCUT Coverage

Companion software
 for Introduction to Software Testing, Ammon and Offutt.
 Copyrighted by Mark A. Beatty, Li Li, Lin Deng, and Scott Brown.
 © 2007–2011. All rights reserved.
 Last update: 05-May-2011

44

For this Semester

- We have posted a temporary version (with permission from the book's authors) that wraps the original app in a python interface (with help from Claude).
- You can use when on campus (or using the Iowa State vpn).
- The app will be taken down on April 28th.
<https://mcc.apps.nimbus.las.iastate.edu/>

45

Coverage Web Application

Companion software to Introduction to Software Testing, Ammann & Offutt.

Compute node, edge, edge-pair, and prime-path coverage test requirements and test paths for directed graphs.

[Open Graph Coverage](#)

Compute DU pairs, DU paths, All-Defs, All-Uses, and All-DU-Paths coverage for data flow graphs.

[Open Data Flow Coverage](#)

Generate truth tables and compute GACC, CACC, and RACC (Active Clause Coverage) test requirements for boolean predicates.

[Open Logic Coverage](#)

Logic Operators

Symbol	Meaning
!	NOT
&	AND
	OR
^	XOR
>	Implies
=	Equivalence

Graph Edge Input Format
Enter each edge on its own line as two node names separated by a space or comma:

```
1 2
2 3
2 4
4 3
```

Data Flow Def/Use Format

```
x 1 2
y 3 1,2
```

Variable name first, then node or edge (e.g. 1,2) definition/uses.

46

Graph Input

Edges
One per line, e.g. 1 2

```
1 3
3 5
5 6
5 7
6 5
```

Initial Node(s)
1
Blank defaults to the first node in the edge list.

Ending Node(s)
7
Space-separated list of ending node names.

Actions

Test Requirements:
[Nodes](#) | [Edges](#) | [Edge Pairs](#) | [Simple Paths](#) | [Prime Paths](#)

Test Paths (Coverage):
[Node Coverage](#) | [Edge Coverage](#) | [Edge Pair Coverage](#) | [Prime Path Coverage](#)

13 test path(s):

```
1. 1 -> 3
2. 3 -> 5
3. 5 -> 6
4. 5 -> 7
5. 6 -> 5
6. 1 -> 3 -> 5
7. 3 -> 5 -> 6
8. 3 -> 5 -> 7
9. 5 -> 6 -> 5
10. 6 -> 5 -> 6
11. 6 -> 5 -> 7
12. 1 -> 3 -> 5 -> 6
13. 1 -> 3 -> 5 -> 7
```

Graph Visualization

47